



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

PROJECT 1

SECP2753

SECTION 02

DATA MINING

LECTURER: DR. ROZILAWATI BINTI DOLLAH @ MD ZAIN

NO.	NAME	MATRIC NUMBER
1.	YASMIN BATRISYIA BINTI ZAHIRUDDIN	A23CS0201
2.	ANIS SAFIYYA BINTI JANAI	A23CS0049
3.	SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183

TABLE OF CONTENT

CONTENT	PAGE
1.0 INTRODUCTION	3
2.0 DATA PREPROCESSING	4
3.0 MODEL DEVELOPMENT	5
3.1 SUPPORT VECTOR MACHINE	5
3.1.1 CLASSIFICATION ALGORITHM FOR INSIGHT NUMBER OF STUDENTS PER GRADE PER DEPARTMENT	5
3.1.2 CLASSIFICATION ALGORITHM FOR INSIGHT AVERAGE TOTAL SCORE BY SLEEP HOURS PER NIGHT	6
3.1.3 CLASSIFICATION ALGORITHM FOR INSIGHT OF AVERAGE TOTAL SCORE AND STRESS LEVEL	8
3.2 RANDOM FOREST	10
3.2.1 CLASSIFICATION ALGORITHM FOR INSIGHT NUMBER OF STUDENTS PER GRADE PER DEPARTMENT	10
3.2.2 CLASSIFICATION ALGORITHM FOR INSIGHT AVERAGE TOTAL SCORE BY SLEEP HOURS PER NIGHT	12
3.2.3 CLASSIFICATION ALGORITHM FOR INSIGHT OF AVERAGE TOTAL SCORE AND STRESS LEVEL	13
3.3 RULE-BASED CLASSIFIERS	15
3.3.1 CLASSIFICATION ALGORITHM FOR INSIGHT NUMBER OF STUDENTS PER GRADE PER DEPARTMENT	15
3.3.2 CLASSIFICATION ALGORITHM FOR INSIGHT AVERAGE TOTAL SCORE BY SLEEP HOURS PER NIGHT	16
3.3.3 CLASSIFICATION ALGORITHM FOR INSIGHT OF AVERAGE TOTAL SCORE AND STRESS LEVEL	18
4.0 MODEL EVALUATION	20

1.0 INTRODUCTION

In this study, we analyze how classification algorithms might be utilized for predicting student performance using a large educational dataset. These variables include department, attendance, midterm and final scores, average assignment and quiz scores, participation, project performance, overall score, study habits, extracurricular involvement, internet access, parental education, family income, stress levels, and sleep duration. The dataset includes a variety of attributes that relate to students' academic records, behavioral patterns, and socioeconomic backgrounds.

This study's main goal is to create predictive models that categorize students into grade groups (A–F) so that teachers and administrators may learn more about the patterns in academic performance in various departments. In order to do this, we apply and contrast three widely used classification methods: Rule-Based Classifiers, K-Nearest Neighbors (KNN), and Support Vector Machines (SVM).

With this project, we hope to:

1. Determine the main factors affecting students' performance.
2. Evaluate the performance of different classification algorithms.
3. Give practical advice on how to improve academic performance and early intervention techniques.

In addition to demonstrating machine learning's potential in the field of education, this analysis highlights the importance of student data in assisting academic institutions in making well-informed decisions.

2.0 DATA PREPROCESSING

Any machine learning project must include data preprocessing to guarantee that the dataset is clean, consistent, and ready for analysis. Most machine learning methods, such as rule-based classifiers, SVM, and K-Nearest Neighbors classification models, need numerical input and are unable to handle categorical or missing data directly. Preprocessing thus helps in converting unprocessed data into a format that these algorithms can understand. In order to preserve data integrity, categorical variables were converted into numerical values in this project, and missing items were handled properly. We enhanced the dataset's quality, minimized potential biases, and guaranteed more accurate and dependable model performance by carrying out these preparation processes.

Here are the example of what we did

- Extracurricular_Activities and Internet_Access_at_Home were converted to numerical values:
 - 1 for YES
 - 0 for NO
- Parent_Education_Level:
 - Missing or empty values were filled with "Unknown"
- Family_Income_Level was mapped to numeric values:
 - 0 for Low
 - 1 for Medium
 - 2 for High
- *Gender* was encoded numerically:
 - 0 for Men
 - 1 for Women

By following these preparation procedures, all categorical data was converted into a numerical representation that could be read by machines and used in classification methods.

In order to maximize model performance before producing significant insights, additional preprocessing—such as feature scaling or dimensionality reduction—may still be necessary, contingent on the particular classification algorithm employed.

3.0 MODEL DEVELOPMENT

This report include 3 classification model which are Support Vector Machine, Random Forest, and Rule-based Classifiers. Each algorithm will be used in classification of insights which are the number of students per grade per department, the average total score by sleep hours per night, and the average total score vs stress level.

3.1 SUPPORT VECTOR MACHINE

Support Vector Machine (SVM) is a supervised machine learning algorithm used for classification and regression tasks. It tries to find the best boundary known as a hyperplane that separates different classes in the data. It is useful when you want to do binary classification like spam vs. not spam or cat vs. dog.

The main goal of SVM is to maximize the margin between the two classes. The larger the margin the better the model performs on new and unseen data.

3.1.1 Classification Algorithm for insight number of students per grade per department

Step 1: Preprocessing data

```
# Drop non-informative columns
df_model = df.drop(columns=["Student_ID", "First_Name", "Last_Name", "Email"])

# Encode categorical columns
label_encoders = {}
categorical_columns = ["Department", "Grade", "Parent_Education_Level"]

for col in categorical_columns:
    le = LabelEncoder()
    df_model[col] = le.fit_transform(df_model[col])
    label_encoders[col] = le
```

The purpose of doing this is to drop a non-informative column that did not contribute to the insight and encode the categorical column. We have to encode the categorical column because SVM and most machine learning models in scikit-learn require numerical input.

Step 2: Split the data into training and testing sets

```
# Split into training and testing sets (70% train, 30% test)
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)
```

In this line, it will splits 70% for training (X_train, y_train) and 30% for testing (X_test, y_test)

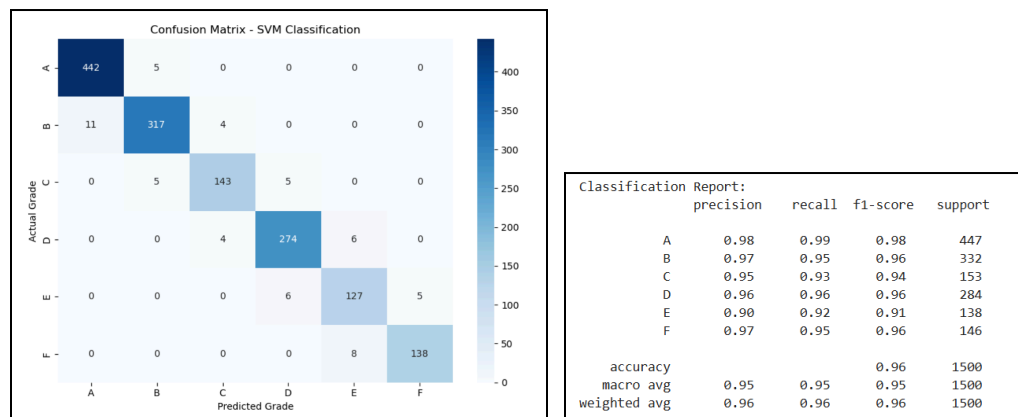
Then, the SVM model is trained on the **training set**:

```
# Train SVM model
svm_model = SVC(kernel='linear', random_state=42)
svm_model.fit(X_train, y_train)
```

And tested (evaluated) on the **testing set**:

```
# Predict and evaluate
y_pred = svm_model.predict(X_test)
```

Step 3: The result



This confusion matrix shows how many predictions were correct or incorrect for each grade.

Each row represents the actual grade, and each column represent the predicted grade. For example, 442 out of 447 actual grade A we correctly predicted as A. then 11 B grades were misclassified as A and 5 C grades were misclassified as B.

As the accuracy in classification report shows 0.96, it means that 96% of total predictions were correct. Therefore the SVM model performs very well, achieving 96% accuracy. Grade A predictions are the most accurate.

3.1.2 Classification Algorithm for insight of average total score by sleep hours per night

Step 1: Preprocessing data

```
# 1. Filter the dataset to only include grades A and D
grades_to_use = ['A', 'D']
df_binary = df[df['Grade'].isin(grades_to_use)]

# 2. Extract features and target
X = df_binary[['Total_Score', 'Sleep_Hours_per_Night']].values
y = df_binary['Grade'].values

# 3. Encode target (A=0, D=1)
le = LabelEncoder()
y_encoded = le.fit_transform(y)
```

Step 2: Split the data into training and testing sets

```
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y_encoded, test_size=0.3, random_state=42)
```

In this line, it will splits 70% for training (X_train, y_train) and 30% for testing (X_test, y_test)

Then, the SVM model is trained on the **training set**:

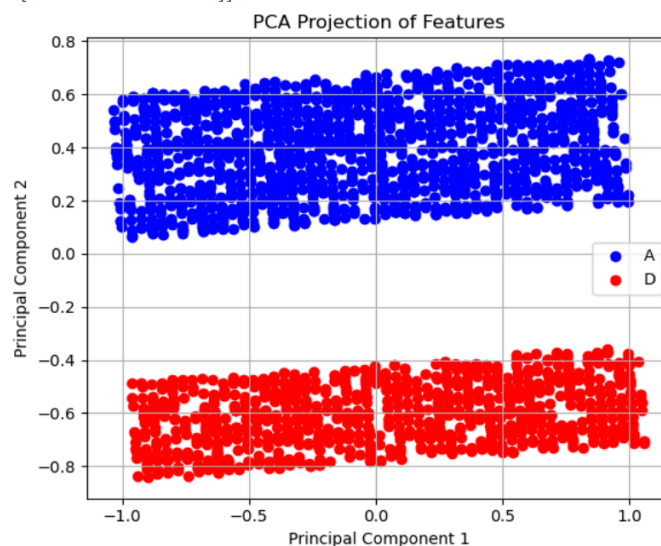
```
model = SVC(kernel='linear')
model.fit(X_train, y_train)
```

And tested (evaluated) on the **testing set**:

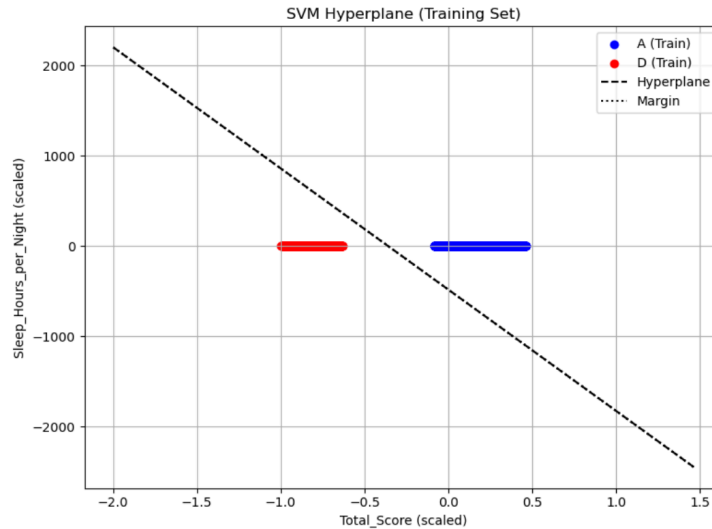
```
y_pred = model.predict(X_test)
print("Accuracy on test set:", accuracy_score(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred, target_names=le.classes_))
```

Step 3: The result

```
Explained variance ratio: [0.56340985 0.43659015]
PCA components (feature contributions):
[[-0.06828038  0.99766617]
 [ 0.99766617  0.06828038]]
```



```
Accuracy on test set: 1.0
```



Classification Report:

	precision	recall	f1-score	support
A	1.00	1.00	1.00	454
D	1.00	1.00	1.00	283
accuracy			1.00	737
macro avg	1.00	1.00	1.00	737
weighted avg	1.00	1.00	1.00	737

Blue dots (grade A) are mostly located on the right-hand side of the plot while Red dots (grade D) are clustered on the left-hand side. This is because students who scored well (grade A) have higher total scores, and those with grade D have lower total scores.

Total scores for D are negative because the data has been standardized or normalized. The student had a negative score because their score is below the mean of the dataset.

Therefore, based on this finding, we can conclude that students with higher scaled total scores and potentially different sleep patterns tend to achieve better grades A with a classification report that says the accuracy is 1.0 (100%)

3.1.3 Classification Algorithm for insight of average total score and stress level

Step 1: Preprocessing data

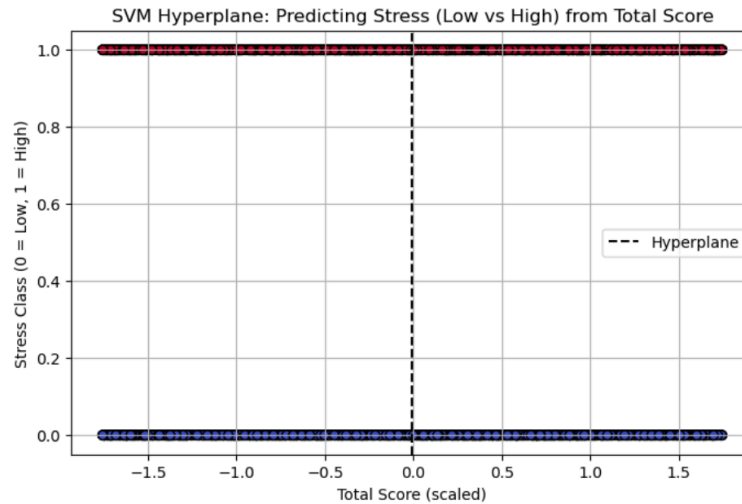
```
df_simple = df_simple[df_simple['Stress_Level (1-10)'].isin([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])]
df_simple['Stress_Class'] = df_simple['Stress_Level (1-10)'].apply(lambda x: 0 if x <= 5 else 1)
```

We do a Binary classification: Low (≤ 5) and high (≥ 6). If the stress level is low, then the stress class is 0 and if the stress level is high, then the stress class is 1.

Step 2: Split the data into training and testing sets

```
# Train/test split
X_train, X_test, y_train, y_test = train_test_split(X, y, stratify=y, test_size=0.3, random_state=42)
```

Step 3: The result



Classification Report:

	precision	recall	f1-score	support
0	0.46	0.47	0.47	749
1	0.47	0.46	0.46	751
accuracy			0.46	1500
macro avg	0.46	0.46	0.46	1500
weighted avg	0.46	0.46	0.46	1500

The model accuracy is only 0.46, which means that the model is correct 46% of the time only. And then, the macro average F-1 Score is 0.46 which indicate poor performance overall, same goes with weighted average F-1 score which is 0.46, which confirms consistent poor performance across both classes.

Therefore, the SVM model performs poorly at predicting stress based on total score.

3.2 RANDOM FOREST

Random Forest is an ensemble machine learning algorithm that builds multiple decision trees during training and combines their results to improve classification accuracy and control overfitting. Each tree is trained on a random subset of the data with random subsets of features, which introduces diversity and reduces the risk of the model memorizing the training data. The final prediction is made by aggregating the predictions from all individual trees. This algorithm is excellent in handling complex, non-linear relationships in data and is robust to noise and outliers.

3.2.1 Classification Algorithm for insight number of students per grade per department

Step 1: Preprocessing the Data

We selected the Department feature as the predictor and Grade as the target variable. Both are categorical and were encoded using LabelEncoder. Missing values were dropped before training.

```
# Step 1: Preprocessing the data
df_rf = df[['Department', 'Grade']]
df_rf.dropna(inplace=True)

# Encode categorical variables
le_dept = LabelEncoder()
le_grade = LabelEncoder()
df_rf['Department'] = le_dept.fit_transform(df_rf['Department'])
df_rf['Grade'] = le_grade.fit_transform(df_rf['Grade'])

X = df_rf[['Department']]
y = df_rf['Grade']
```

We selected only the relevant columns (Department and Grade) because this insight is about the relationship between department and grade. We then dropped the missing values using `dropna()` to ensure the model doesn't encounter errors or bias due to incomplete data.

Since Random Forest cannot process text/categorical data directly, we used `LabelEncoder()` to convert text labels into numeric codes to make sure the data is usable by the model. EG, departments like “CS”, “Math”, “Physics” become 0,1,2, etc.

Step 2: Train-test split

We split the dataset (70% train, 30% test) and trained a Random Forest Classifier on the encoded department feature.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
```

The training set is used to build the model, test set is used to evaluate the model's predictive performance.

Step 3: Train model

```
# Step 3: Train model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
```

Step 4: Evaluation

```
from sklearn.metrics import classification_report, confusion_matrix

y_pred = rf_model.predict(X_test)

print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test, y_pred))
```

```
[[112 123   0   0 114  98]
 [ 86  84   0   0  83  79]
 [ 33  47   0   0  35  38]
 [ 69  72   0   0  69  74]
 [ 29  31   0   0  39  39]
 [ 36  48   0   0  31  31]]
```

	precision	recall	f1-score	support
0	0.31	0.25	0.28	447
1	0.21	0.25	0.23	332
2	0.00	0.00	0.00	153
3	0.00	0.00	0.00	284
4	0.11	0.28	0.15	138
5	0.09	0.21	0.12	146
accuracy			0.18	1500
macro avg	0.12	0.17	0.13	1500
weighted avg	0.16	0.18	0.16	1500

Based on the result above, the classifier performed poorly. Accuracy was 18%, grades C and D were not predicted, grades A and B were frequently confused with other labels. This is because Department alone is not a strong predictor of student performance.

3.2.2 Classification Algorithm for insight Average total score vs Sleep mper hours per night.

Step 1: Preprocessing the Data

We select only the relevant columns Total_Score and Stress_Level (1-10) because these are the variables needed to predict stress levels based on students' scores. We also remove rows with missing values using `.dropna()` to ensure the model trains on complete data and avoids errors caused by missing information.

```
# Prepare dataset
df_clf = df[['Total_Score', 'Stress_Level (1-10)']].dropna()

# Encode target labels
le_stress = LabelEncoder()
df_clf['Stress_Level (1-10)'] = le_stress.fit_transform(df_clf['Stress_Level (1-10)'])

X = df_clf[['Total_Score']]
y = df_clf['Stress_Level (1-10)']
```

To proceed with the training of the model, we must specify what the inputs (features) and outputs (targets) are. Here, Total_Score is the input feature we will use to predict the output, which is the stress level (Stress_Level (1-10)). Separating them clearly helps the training process understand what to learn from and what to predict.

Step 2: Train-test split

The dataset is divided into two parts: 70% for training the model and 30% for testing its performance. The training set is used by the Random Forest Classifier to learn patterns between the features and the target. The test set is kept separate and used later to assess how well the model can predict stress levels on new, unseen data. This helps ensure that the model generalizes well and is not just memorizing the training data.

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 3: Train model

```
# Train model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
```

Step 4: Evaluation

```
#evaluation|
y_pred = rf_model.predict(X_test)

print(classification_report(y_test, y_pred, target_names=[str(c) for c in le_stress.classes_]))
```

	precision	recall	f1-score	support
1	0.07	0.06	0.06	161
2	0.13	0.10	0.12	164
3	0.11	0.10	0.10	145
4	0.15	0.14	0.14	155
5	0.10	0.11	0.10	146
6	0.08	0.09	0.08	140
7	0.10	0.10	0.10	154
8	0.11	0.10	0.10	174
9	0.07	0.09	0.08	129
10	0.11	0.14	0.12	132
accuracy			0.10	1500
macro avg	0.10	0.10	0.10	1500
weighted avg	0.10	0.10	0.10	1500

The Random Forest model's accuracy for predicting stress levels (1–10) is low at 10%. Precision, recall, and f1-scores are all below 0.15 for most classes, indicating poor prediction performance. This suggests that Total_Score alone is insufficient to accurately predict stress levels, and more features or better models are needed.

3.2.3 Classification Algorithm for insight Average total score vs Stress level.

Step 1: Preprocessing the Data

Select only the relevant columns, clean missing values, and encode the target variable (Sleep Hours per Night) as a classification target.

```
# Prepare dataset with relevant columns
df_clf_sleep = df[['Total_Score', 'Sleep_Hours_per_Night']].dropna()

# Define bins and labels for sleep hours
bins = [0, 5, 7, 9, 12] # Adjust bins as needed
labels = ['Very Low', 'Low', 'Moderate', 'High']

# Bin the sleep hours into categories
df_clf_sleep['Sleep_Hours_Cat'] = pd.cut(df_clf_sleep['Sleep_Hours_per_Night'], bins=bins, labels=labels, include_lowest=True)

# Encode the sleep categories
le_sleep = LabelEncoder()
df_clf_sleep['Sleep_Hours_Cat_Encoded'] = le_sleep.fit_transform(df_clf_sleep['Sleep_Hours_Cat'].astype(str))

# Features and target
X = df_clf_sleep[['Total_Score']]
y = df_clf_sleep['Sleep_Hours_Cat_Encoded']
```

This step filters the dataset to only the columns relevant for this insight — total score and sleep hours per night. We remove missing data to avoid errors. If the sleep hours column is categorical, we encode it into numeric labels so the model can process it.

Step 2: Train-test split

We separate the independent variables here, the total score from the dependent variable, which is the sleep hours per night category.

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

Step 3: Train model

We split the data so 70% is used to train the model, and 30% to test its performance on unseen data. The random state ensures reproducibility.

```
# Train model
rf_model = RandomForestClassifier(random_state=42)
rf_model.fit(X_train, y_train)
```

Step 4: Evaluation

```
# Predict on test data
y_pred = rf_model.predict(X_test)

print(classification_report(y_test, y_pred, target_names=le_sleep.classes_))
```

	precision	recall	f1-score	support
Low	0.38	0.41	0.39	576
Moderate	0.39	0.38	0.39	602
Very Low	0.24	0.22	0.23	322
accuracy			0.36	1500
macro avg	0.34	0.34	0.34	1500
weighted avg	0.35	0.36	0.35	1500

The Random Forest model achieved 36% accuracy in classifying students' sleep hours based on total scores. It performed moderately for "Low" and "Moderate" sleep categories, with F1-scores of 39% each. However, it struggled with the "Very Low" category, achieving only a 23% F1-score. This suggests that total score alone may not strongly predict sleep hour categories.

3.3 RULE-BASED CLASSIFIERS

Rule-based classifiers are the most popular machine learning classification technique. This technique uses a set of rules, which are often called “if-then”, to categorize data into pre-defined groups or categories. The rule-based classifications are good as they create consistency in making decisions and can be seen in real-world applications such as to classify patient symptoms in medical diagnosis. This classifier are also known as decision tree analysis, and it recursively divides a training set until each division consist primarily of examples from one class.

3.3.1 Classification Algorithm for insight number of students per grade per department

Step 1: Preprocessing data

```
from sklearn.preprocessing import LabelEncoder

df.drop(columns=['Student_ID', 'First_Name', 'Last_Name', 'Email'], inplace=True, errors='ignore')

label_encoders = {}

for column in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le
```

The purpose of doing this is to drop a non-informative column that did not contribute to the insight and encode the categorical column. The data also needs to be encoded in order to transform into a format suitable for machine learning

Step 2: Split the data into training and testing sets

```
X = df.drop(columns=['Grade'])
y = df['Grade']

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
```

The data will then be split into a training and a testing set, with 70% and 30% each from the dataset.

Step 3: Decision tree

```
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
```

Using the rule-based algorithm to predict student grades based on features like department.

Step 4: The result

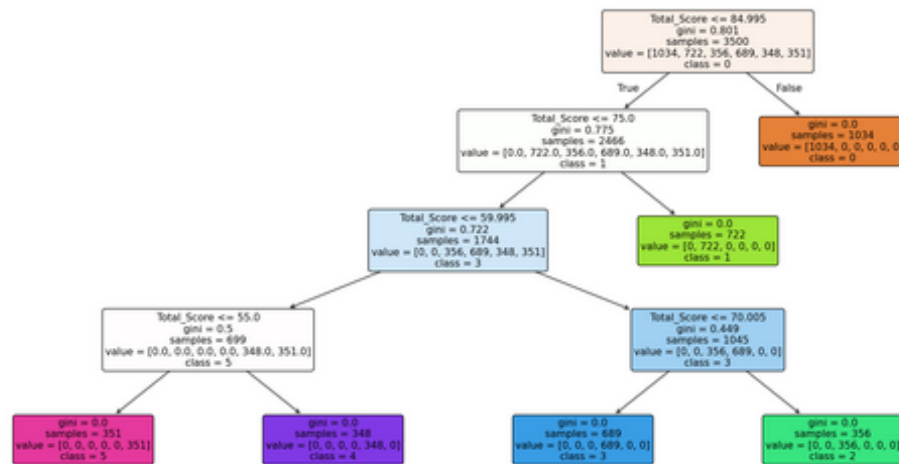
Accuracy: 0.998

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	447
1	1.00	0.99	1.00	332
2	0.99	0.99	0.99	153
3	1.00	1.00	1.00	284
4	1.00	1.00	1.00	138
5	1.00	1.00	1.00	146
accuracy			1.00	1500
macro avg	1.00	1.00	1.00	1500
weighted avg	1.00	1.00	1.00	1500

Confusion Matrix:

```
[[447  0  0  0  0  0]
 [  0 330  2  0  0  0]
 [  0  0 152  1  0  0]
 [  0  0  0 284  0  0]
 [  0  0  0  0 138  0]
 [  0  0  0  0  0 146]]
```



Based on the classification report and the confusion matrix, the classifier achieved an accuracy of 99.8%, which means that it correctly predicted nearly all of the student grade categories in the test set.

3.3.2 Classification Algorithm for insight of average total score by sleep hours per night

Step 1: Preprocessing data


```

from sklearn.preprocessing import LabelEncoder

df.drop(columns=['Student_ID', 'First_Name', 'Last_Name', 'Email'], inplace=True, errors='ignore')

label_encoders = {}

for column in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le

```

The purpose of doing this is to drop a non-informative column that did not contribute to the insight and encode the categorical column. The data also needs to be encoded in order to transform into a format suitable for machine learning

Step 2: Split the data into training and testing sets

```

X = df.drop(columns=['Grade'])
y = df['Grade']

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

```

Step 3: Decision tree

```

clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)

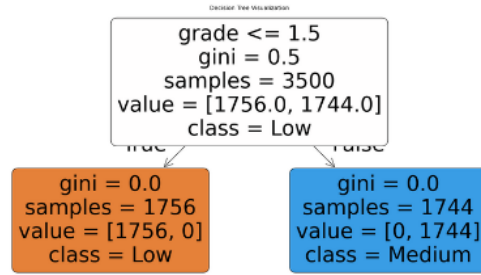
```

Step 4: The result

Accuracy: 0.9986666666666667

Classification Report:

	precision	recall	f1-score	support
0	1.00	1.00	1.00	777
1	1.00	1.00	1.00	723
accuracy			1.00	1500
macro avg	1.00	1.00	1.00	1500
weighted avg	1.00	1.00	1.00	1500



The decision tree classifier showed exceptional results in predicting the target categories, as seen by its unusually high accuracy of 99.87%. In the test set of 1,500 samples, the precision, recall, and F1-scores for both classes (0 and 1) are all 1.00, indicating flawless classification performance with neither false positives nor false negatives. Such near-perfect results may also call for a check for possible overfitting, particularly if the model has access to extremely detailed patterns in the training data. This level of performance indicates that the model fits the data very well. The evaluation's overall findings support the classifier's outstanding accuracy on this dataset.

3.3.3 Classification Algorithm for insight Average Total Score vs Stress Level.

Step 1: Preprocessing data

```

from sklearn.preprocessing import LabelEncoder

df.drop(columns=['Student_ID', 'First_Name', 'Last_Name', 'Email'], inplace=True, errors='ignore')

label_encoders = {}

for column in df.select_dtypes(include=['object']).columns:
    le = LabelEncoder()
    df[column] = le.fit_transform(df[column])
    label_encoders[column] = le
  
```

The purpose of doing this is to drop a non-informative column that did not contribute to the insight and encode the categorical column. The data also needs to be encoded in order to transform into a format suitable for machine learning

Step 2: Split the data into training and testing sets

```

X = df.drop(columns=['Grade'])
y = df['Grade']

# Split the dataset
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
  
```

Step 3: Decision tree

```

clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
  
```

Step 4: The result

```
Accuracy: 0.10533333333333333
Classification Report:
              precision    recall  f1-score   support

     1         0.16         0.16         0.16         161
     2         0.11         0.11         0.11         164
     3         0.13         0.13         0.13         145
     4         0.09         0.08         0.09         155
     5         0.11         0.12         0.11         146
     6         0.08         0.07         0.07         140
     7         0.09         0.09         0.09         154
     8         0.14         0.11         0.12         174
     9         0.08         0.09         0.08         129
    10         0.07         0.08         0.08         132

 accuracy          0.11         0.11         0.11         1500
  macro avg         0.11         0.10         0.10         1500
 weighted avg         0.11         0.11         0.11         1500

Confusion Matrix:
[[25 16 16 16 13 14 16 15 17 13]
 [13 18 19 21 17 12 22 10 15 17]
 [18 10 19 13 12 12 15 13 12 21]
 [14 15 17 13 21 13 16 12 14 20]
 [16 18 10  9 18 19 18 12 11 15]
 [12 17 12 16 21 10 10 12 17 13]
 [12 12 21 12 23 10 14 13 21 16]
 [20 26  9 18 20 15 20 19 10 17]
 [14 14 10  8 13 14 20 11 11 14]
 [12 18 15 15 12  8 12 17 12 11]]
```

The decision tree classifier's performance in predicting students' stress levels based on average total score is very poor, with an overall accuracy of approximately 10.5%. The classification report shows uniformly low precision, recall, and F1-scores for all stress level categories, indicating the model struggles to correctly identify any specific stress level. The confusion matrix further supports this by revealing a scattered prediction pattern, with misclassifications distributed across all classes. This suggests that the current features do not have strong discriminatory power for predicting stress levels.

4.0 MODEL EVALUATION

4.1 Insight number of students per grade per department

	Accuracy	Precision	Recall	F-1 score
SVM	0.96	0.96	0.96	0.96
Random Forest	0.18	0.16	0.18	0.16
Rule-based classifiers	0.99	1.0	1.0	1.0

The table shows that the rule-based classifiers have the best performance with an accuracy of 0.99. It also gain a perfect score of 1.0 for the precision, recall, and f-1 score. Next, the SVM classifiers also shows an excellence performance with a score of 0.96 for all of the measures. Thus, both SVM and rule-based classifiers are suitable for gaining insights on the number of students per grade per department. Meanwhile, the Random Forest is not suitable for this insight as it shows a poor result of 0.16 to 0.18 across the measures.

4.2 Insight of average total score by sleep hours per night

	Accuracy	Precision	Recall	F-1 score
SVM	1.0	1.0	1.0	1.0
Random Forest	0.10	0.10	0.10	0.10
Rule-based classifiers	1.0	1.0	1.0	1.0

Both the SVM and rule-based classifiers achieved perfect scores (1.0) on all measures, meaning they classified every instance accurately and free of errors.

On the other hand, Random Forest only managed to achieve 10% in each statistic, showing that it was mainly ineffective for this specific purpose.

For this classification task, the SVM and Rule-based classifiers perform very well, producing excellent results. This means that the rules perfectly represent the desired logic or that the data may be linearly separable. On the other hand, the Random Forest model is not suited in this situation, whether as a result of incorrect hyperparameters, underfitting, or overfitting.

4.3 Insight average total score vs stress level.

	Accuracy	Precision	Recall	F-1 Score
SVM	0.46	0.46	0.46	0.46
Random Forest	0.36	0.35	0.36	0.35
Rule-based classifiers	0.10	0.11	0.11	0.11

Based on the comparison, the SVM is better than Random Forest and Rule-based Classifiers in gaining insights on the relationship of the average total score with stress level. The SVM algorithm obtains an accuracy, precision, recall and F-1 score of 0.46 which was the highest than the other two algorithms. This suggests that SVM was able to capture the patterns between the input features and the stress level classes compared to other models.

The following model are the Random Forest with moderate score of 0.35 to 0.36 across the metrics. Next, the rule-based classifiers shows a poor performance with an accuracy of 0.10, and also 0.11 for the other metrics. It is implying that it failed to learn meaningful decision rules from the data in this specific task.

5.0 Conclusion

In this project, we investigated the performance of three different classification algorithms which are Support Vector Machine(SVM), Random Forest and Rule-Based Classifiers applied to the student data with the key insights being the number of students per grade per department, average total score by sleep hours and average total score vs stress level. Based on the results, we can conclude that SVM and rule-based models work well for structured, linear relationships. Random Forest needs better tuning and more features to perform competitively.

In conclusion, to optimize the usage of the classification algorithms in the future, we need to choose models based on the data characteristics as the suitability of a classifier depends greatly on the structure and separability of the underlying data.

6.0 References

Dataset used: <https://www.kaggle.com/datasets/mahmoudelhemy/students-grading-dataset>

GeeksforGeeks. (n.d.). *Random Forest algorithm in Machine Learning*. GeeksforGeeks. <https://www.geeksforgeeks.org/random-forest-algorithm-in-machine-learning/>

Hashemi, A. (n.d.). *Random Forest classification algorithm*. Kaggle.
<https://www.kaggle.com/code/hashemialii/random-forest-classification-algorithm>

GeeksforGeeks. (n.d.). *Support Vector Machine Algorithm*. GeeksforGeeks.
<https://www.geeksforgeeks.org/support-vector-machine-algorithm/>

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, É. (2011). *Scikit-learn: Machine learning in Python*. Journal of Machine Learning Research, 12, 2825–2830. <https://scikit-learn.org/stable/modules/svm.html>

GeeksforGeeks. (n.d.). *Rule-Based Classifier in Machine Learning*. GeeksforGeeks.
<https://www.geeksforgeeks.org/rule-based-classifier-machine-learning/>

TutorialsPoint. (n.d.). *Data Mining - Rule Based Classification*. TutorialsPoint.
https://www.tutorialspoint.com/data_mining/dm_rbc.htm